

01 / PROJECT OBJECTIVE

Distributed File System

Inspired by the Google File System architecture. A client-based system for text-oriented files that hides physical distribution behind a clean file interface.



Sharding

Fixed-size chunks distributed across machines



Replication

Each chunk stored on 3 distinct servers



Separation

Metadata layer independent from content

Three-Layer Architecture

> Client

- > Requests metadata from naming server
- > Transfers chunk data directly to/from storage
- > Provides create, read, delete, size, ls

Naming Server

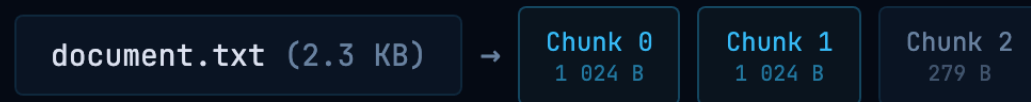
- › Single metadata authority
- › Tracks files, chunks, replica locations
- › SQLite persistence — survives restart
- › Never stores chunk content

 Storage Servers (x4)

- › Store chunk bytes as local files
- › Each holds a subset of total chunks
- › Register + send heartbeats every 5s

Chunks, UUIDs & Replication

FILE → CHUNK PARTITIONING



UUID CHUNK IDENTIFIER

```
chunk_id: a3f8b2c194e04d71b3a21f0c8e5d7a29
```

REPLICATION FACTOR 3 – ROUND-ROBIN PLACEMENT

Configuration

Workload	Text-oriented files
Chunk size	1 024 bytes fixed
Last chunk	May be smaller
Identifier	UUID per chunk
Replication	Factor 3
Placement	Round-robin (live)
SQLite	Metadata only
Content	Disk files on servers

Storage separation

Only metadata is in SQLite. Chunk bytes live as regular files under each storage server's `/data` directory — satisfying the required metadata/content separation.

Split

Client splits the input into 1 KB chunks and calculates total file size.

2 Request placement

3 Upload replicas

4 Commit

Metadata state machine



Sequence

```
client → naming: CreateFile(name,
size)
naming → client: placement plan
client → S1/S2/S3: StoreChunk(uuid,
data)
S* → client: ack
client → naming: CommitFile(name)
```

Atomicity guarantee

Storage servers write to a temp file before renaming atomically. No partial chunk is ever exposed.

Client-Side Operations



Read

- 01 Client requests chunk layout + replica addresses from naming server
- 02 Downloads each chunk directly from a storage server
- 03 Concatenates chunks in index order to reconstruct file

Replica fallback

- S1

● S2

● S3
- try next

success

not needed

Naming server prioritizes live replicas in its response.



Delete

- 01 Naming server requests deletion of every known chunk replica
- 02 Then removes the file metadata entry from SQLite

Idempotent

Chunk deletion calls are safe to repeat.

Orphan chunks

If a server is unavailable, its copy becomes an unreferenced orphan — unreachable through the file system but not yet removed. Garbage collection is future work.



Size & List

size(filename)

Answered exclusively from metadata. Returns total byte count and number of chunks — **no chunk data is transferred**.

→ { bytes: 2359, chunks: 3 }

ls()

Convenience operation. Lists all committed files in the namespace.

document.txt
report.md
notes.txt

Resilience Model

FAILURE SCENARIOS (RF=3, 4 SERVERS)

- 1 server down

2 servers down

- All 3 replicas lost

● Naming server down

AUTOMATIC SELF-HEALING

Re-replication sequence

- 1 Heartbeat every 5s; server marked unavailable after 15s silence
- 2 Every 5s: naming server scans committed chunks for under-replication
- 3 Chunk with < 3 live replicas + enough live capacity $\rightarrow$ trigger repair
- 4 Choose surviving source + target server (doesn't hold chunk)
- 5 Target retrieves bytes via ReplicateChunk RPC from source
- 6 Metadata updated only after copy succeeds — RF restored silently

RF restored automatically — zero client intervention required.

Docker, Observability & Tests

DOCKER COMPOSE TOPOLOGY

naming	50051 host	Metadata authority
storage1	50061 int.	Replica store
storage2	50061 int.	Replica store
storage3	50061 int.	Replica store
storage4	50061 int.	Replica store
client	Helper container	

All containers share one internal Docker network. Only the naming-server port is exposed to host clients.

CLIENT COMMANDS

<code>\$ python -m gfs.client create <local> [remote]</code>	Upload and shard a file
<code>\$ python -m gfs.client read <remote> [out]</code>	Download and reconstruct
<code>\$ python -m gfs.client delete <remote></code>	Remove replicas + metadata
<code>\$ python -m gfs.client size <remote></code>	Metadata-only byte count
<code>\$ python -m gfs.client ls</code>	List committed files

INTEGRATION TEST SUITE (6 SCENARIOS)

- **T1** Create + read correctness (multi-chunk file)
- **T2** Metadata-only size query
- **T3** Read after one storage-server failure
- **T4** Replication factor restoration via self-healing
- **T5** Deletion of file and replicas
- **T6** Reject create when insufficient servers alive

Monitoring & simulation

Prometheus and Grafana expose live servers, files, bytes, RPC latency/throughput, storage usage, and healing signals. The production simulator drives configurable users, file sizes, and write/read/delete ratios.

Trade-offs & Summary

Principal design decisions

- > **Single naming server** — Simple, consistent metadata without coordination overhead
- > **SQLite persistence** — Durable metadata — naming server survives process restart
- > **Direct client↔storage** — Bulk data bypasses naming server — no bottleneck
- > **Synchronous replication** — All replicas confirmed before commit — strong durability
- > **Fixed 1 KB chunks** — Required by specification; enables uniform placement

Known limitations

Conclusion

- ✓ Files partitioned into fixed-size chunks
- ✓ Each chunk replicated across distinct storage servers
- ✓ Metadata separated from content
- ✓ Client hides physical distribution
- ✓ Create, read, delete, size, and ls implemented
- ✓ Automatic self-healing restores lost replicas

The system demonstrates both immediate failover through replication and subsequent recovery through automatic re-replication.

FINAL CAPABILITIES

Sharding	PASS
Replication (RF=3)	PASS
Metadata separation	PASS
Fault tolerance	PASS
Self-healing	PASS
Docker deployment	PASS
Observability	PASS
Integration tests	PASS